Facultad de Ingenieria, Industria y Tecnologia
Estructura de Datos

EXAMEN PARCIAL PRACTICO

Algoritmos Iterativos vs. Recursivos

Analisis de Complejidad Temporal y Notacion Big-O

Catedratico:	Ing. Brandon Chitay
Asignatura:	Estructura de Datos
Facultad:	Ingenieria, Industria y Tecnologia
Tipo de evaluacion:	Examen Parcial Practico
Valor total:	100 puntos
Modalidad:	Individual
Lenguaje:	Java (version 11 o superior)
Tiempo de entrega:	1 semana a partir de la fecha de asignacion

Celeste Fuentes Sepet

Carne: 202500185

INTRODUCCION

Un algoritmo es un conjunto de instrucciones que sirven para generar una salida, entre ellos tenemos a:

- Iterativos (resuelve problemas utilizando bucles, mantiene el estado en variables y repite los pasos hasta terminar) uso de memoria bajo, suele ser más rápido y apto para procesos lineales
- Recursivos (es la función que se llama a si misma para resolver subproblemas, hasta alcanzar el caso base) uso de memoria mayor, apto para árboles, divisiones, se basa en los casos base

La complejidad temporal mide cuánto tiempo tarda un algoritmo en ejecutarse cuando crece el tamaño de su entrada.

La notación Big O es una forma sencilla de decir cómo crece el tiempo o la memoria que necesita un algoritmo cuando aumenta el tamaño de los datos. Se fija en el comportamiento cuando los datos son muy grandes e ignora constantes y detalles pequeños, lo cual nos es de mucha ayuda a la hora de estimar el futuro de nuestras aplicaciones.

Big O pertenece a la notación asintótica, y nos ayuda a:

- Comparar algoritmos, para saber que escala nos conviene mas
- Predecir rendimiento, estima si habrá una solución practica para la entrada de datos grandes

Tenemos:

- Big O(1) el constante, no importa cuanto crezcan el tiempo no cambia
- Big O(n) el lineal, el tiempo crece dependiendo de la cantidad de elementos
- Big O(n^2) el cuadráticos, el tiempo crece mucho más rápido normalmente dos bucles anidados
- Big O(log n) logarítmico, crece muy despacio, su filosofía es la división
- El que es optimo para entradas grandes O(n)

DESCRIPCIÓN DE ALGORITMOS

FACTORIAL ITERATIVO

Calcula n multiplicado sucesivamente los enteres desde 1 hasta n. Es simple y eficiente, lo malo es que no aprovecha la recursividad

```java
public class Factorial {

    // ------------------------------------------------------------
    // VERSIÓN ITERATIVA
    // ------------------------------------------------------------
    /**
     * Calcula el factorial de n usando un ciclo.
     *
     * @param n número entero (n ≥ 0)
     * @return n!
     */
    public static long iterativo(int n) {
        if (n < 0) throw new IllegalArgumentException(s: "n no puede ser negativo");

        long resultado = 1;
        for (int i = 2; i <= n; i++) {
            resultado *= i;
        }
        return resultado;
    }
```

FACTORIAL RECURSIVO

Se define por el caso base 0!=1 y relación n!=n*(n-1) La función se llama a si misma hasta alcanzar el caso base, usa espacio adicional por pila de llamadas, tiene riesgo de stack u overflow

```java
// ------------------------------------------------------------
// VERSIÓN RECURSIVA
// ------------------------------------------------------------
/**
 * Calcula el factorial de n usando recursión directa.
 *
 * Fórmula recursiva:
 *   fact(0) = 1              ← caso base
 *   fact(n) = n × fact(n-1)  ← llamada recursiva
 *
 * @param n número entero (n ≥ 0)
 * @return n!
 */
public static long recursivo(int n) {
    if (n < 0) throw new IllegalArgumentException(s: "n no puede ser negativo");

    // --- CASO BASE ---
    if (n == 0) return 1;

    // --- LLAMADA RECURSIVA ---
    return n * recursivo(n - 1);
}
```

FIBONACCI ITERATIVO

Calcula el n-esimo termino de la serie usando un bucle y dos variables, evita la explosión de llamadas recursivas, es muy eficiente

```java
// VERSIÓN ITERATIVA
// ----------------------------------------------------------------
/**
 * Calcula el n-ésimo Fibonacci con dos variables deslizantes.
 *
 * Traza para n = 6:
 *   anterior=0, actual=1
 *   i=2 → sig=1,  anterior=1, actual=1
 *   i=3 → sig=2,  anterior=1, actual=2
 *   i=4 → sig=3,  anterior=2, actual=3
 *   i=5 → sig=5,  anterior=3, actual=5
 *   i=6 → sig=8,  anterior=5, actual=8  ← retorna 8
 *
 * @param n índice del término (0-indexado)
 * @return el n-ésimo número de Fibonacci
 */
public static long iterativo(int n) {
    if (n < 0) throw new IllegalArgumentException(s: "n no puede ser negativo");

    if (n == 0) return 0;
    if (n == 1) return 1;

    long anterior = 0;
    long actual   = 1;

    for (int i = 2; i <= n; i++) {
        long siguiente = anterior + actual;
        anterior = actual;
        actual   = siguiente;
    }
    return actual;
}
```

FIBONACCI RECURSIVO

Define F(n)=F(n-1) + F(n-2) con casos base F(0)=0,F(1)=1, llama a la función dos veces por nivel de memorización, requiere mas tiempo y espacio adicional por la pila de llamadas, es extremadamente ineficiente

```
// ----------------------------------------------------------------
// VERSIÓN RECURSIVA (sin memoización — intencional para análisis)
// ----------------------------------------------------------------
/**
 * Calcula el n-ésimo Fibonacci con recursión directa.
 *
 * Fórmula recursiva:
 *   fib(0) = 0              ← caso base
 *   fib(1) = 1              ← caso base
 *   fib(n) = fib(n-1) + fib(n-2)  ← dos llamadas recursivas
 *
 * Árbol de llamadas para n = 5:
 *             fib(5)
 *           /         \
 *        fib(4)       fib(3)
 *       /     \       /    \
 *    fib(3) fib(2) fib(2) fib(1)
 *     ...
 *
 * Nota: fib(3) se calcula DOS VECES → trabajo duplicado = O(2^n)
 *
 * @param n índice del término (0 ≤ n ≤ 30 para tiempos razonables)
 * @return el n-ésimo número de Fibonacci
 */
public static long recursivo(int n) {
    if (n < 0) throw new IllegalArgumentException(s: "n no puede ser negativo");
    if (n > 30) throw new IllegalArgumentException(s: "Limita n ≤ 30 en versión rec

    // --- CASOS BASE ---
    if (n == 0) return 0;
    if (n == 1) return 1;

    // --- DOS LLAMADAS RECURSIVAS = crecimiento exponencial ---
    return recursivo(n - 1) + recursivo(n - 2);
}
```

BUSQUEDA LINEAL ITERATIVA

Recorre el arreglo elemento a elemento hasta encontrar el objetivo; devuelve el índice o -1 si no existe, siempre funciona en arreglos no ordenados, no es escalable

```java
package algorithms;

public class BusquedaLineal {

    // ----------------------------------------------------------------
    // VERSIÓN ITERATIVA
    // ----------------------------------------------------------------
    /**
     * Busca un valor en el arreglo recorriéndolo secuencialmente.
     *
     * @param arr arreglo de enteros
     * @param objetivo valor a buscar
     * @return índice del valor si existe, -1 si no se encuentra
     */
    public static int iterativo(int[] arr, int objetivo) {
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == objetivo) {
                return i; // índice encontrado
            }
        }
        return -1; // no encontrado
    }
```

```java
// ---- BÚSQUEDA LINEAL ITERATIVO ----
System.out.println(x: "\n  BÚSQUEDA LINEAL ITERATIVO");
Medidor.imprimirEncabezado();
for (int objetivo : arregloDesordenado) {
    int resultado = BusquedaLineal.iterativo(arregloDesordenado, objetivo);
    double tiempoMs = Medidor.medir(() -> BusquedaLineal.iterativo(arregloDesordenado, objetivo));
    Medidor.imprimirFila(algoritmo: "BusquedaLineal", version: "Iterativo", objetivo, tiempoMs);
    csv.append(String.format(format: "BusquedaLineal,Iterativo,%d,%d,%.6f%n", objetivo, resultado, tiempoMs));
}

// ---- BÚSQUEDA LINEAL RECURSIVA ----
System.out.println(x: "\n  BÚSQUEDA LINEAL RECURSIVA");
Medidor.imprimirEncabezado();
for (int objetivo : arregloDesordenado) {
    int resultado = BusquedaLineal.recursivo(arregloDesordenado, objetivo, indice: 0);
    double tiempoMs = Medidor.medir(() -> BusquedaLineal.recursivo(arregloDesordenado, objetivo, indice: 0));
    Medidor.imprimirFila(algoritmo: "BusquedaLineal", version: "Recursivo", objetivo, tiempoMs);
    csv.append(String.format(format: "BusquedaLineal,Recursivo,%d,%d,%.6f%n", objetivo, resultado, tiempoMs));
}
```

BUSQUEDA LINEAL RECURSIVA

Hace la misma comprobación, pero avanzando el índice mediante llamadas recursivas, utiliza espacio adicional por la pila de llamadas (el peor caso), es claro, por su uso de memoria no aporta ventaja

```java
// ------------------------------------------------------------
// VERSIÓN RECURSIVA
// ------------------------------------------------------------
/**
 * Busca un valor en el arreglo usando recursión.
 *
 * @param arr arreglo de enteros
 * @param objetivo valor a buscar
 * @param indice posición actual (inicia en 0)
 * @return índice del valor si existe, -1 si no se encuentra
 */
public static int recursivo(int[] arr, int objetivo, int indice) {
    // --- CASO BASE ---
    if (indice >= arr.length) {
        return -1; // no encontrado
    }
    if (arr[indice] == objetivo) {
        return indice; // encontrado
    }

    // --- LLAMADA RECURSIVA ---
    return recursivo(arr, objetivo, indice + 1);
}
```

ORDENAMIENTO BURBUJA ITERATIVA

Repite pasadas por el arreglo intercambiando elementos adyacentes fuera de orden; tras cada pasada el mayor elemento “burbujea” al final, su tiempo esta basado en promedio y peor caso, es estable y fácil de implementar, es ineficiente para arreglos grandes

```java
package algorithms;

public class Burbuja {

    // ------------------------------------------------------------
    // VERSIÓN ITERATIVA
    // ------------------------------------------------------------
    public static int[] iterativo(int[] arr) {
        int[] copia = arr.clone(); // para no modificar el original
        int n = copia.length;
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                if (copia[j] > copia[j + 1]) {
                    int temp = copia[j];
                    copia[j] = copia[j + 1];
                    copia[j + 1] = temp;
                }
            }
        }
        return copia;
    }
```

```java
// ---- BURBUJA ITERATIVO ----

    // ---- BURBUJA ITERATIVO ----
System.out.println(x: "\n  BURBUJA ITERATIVO  [O(n^2)]");
Medidor.imprimirEncabezado();
for (int objetivo : arregloDesordenado) {
    double tiempoIter = Medidor.medir(() -> Burbuja.iterativo(arregloDesordenado));
    Medidor.imprimirFila(algoritmo: "Burbuja", version: "Iterativo", objetivo, tiempoIter);
    csv.append(String.format(format: "Burbuja,Iterativo,%d,%.6f%n",
            objetivo,
            tiempoIter));
}

// ---- BURBUJA RECURSIVO ----
System.out.println(x: "\n  BURBUJA RECURSIVO  [O(n^2)]");
Medidor.imprimirEncabezado();
for (int objetivo : arregloDesordenado) {
    double tiempoRec = Medidor.medir(() -> Burbuja.recursivo(arregloDesordenado, arregloDesordenado.length));
    Medidor.imprimirFila(algoritmo: "Burbuja", version: "Recursivo", objetivo, tiempoRec);
    csv.append(String.format(format: "Burbuja,Recursivo,%d,%.6f%n",
            objetivo,
            tiempoRec));
}
```

BURBUJA RECURSIVO

Realiza una pasada de burbuja para mover el mayor al final y luego llama recursivamente para ordenar los primeros n-1, suele tener overhead de llamadas, no recomendado

```java
// ----------------------------------------------------------------
// VERSIÓN RECURSIVA
// ----------------------------------------------------------------
public static int[] recursivo(int[] arr, int n) {
    int[] copia = arr.clone(); // para no modificar el original
    recursivoHelper(copia, n);
    return copia;
}

private static void recursivoHelper(int[] arr, int n) {
    // Caso base
    if (n == 1) return;

    // Una pasada de burbuja
    for (int i = 0; i < n - 1; i++) {
        if (arr[i] > arr[i + 1]) {
            int temp = arr[i];
            arr[i] = arr[i + 1];
            arr[i + 1] = temp;
        }
    }

    // Llamada recursiva para el resto
    recursivoHelper(arr, n - 1);
}
```

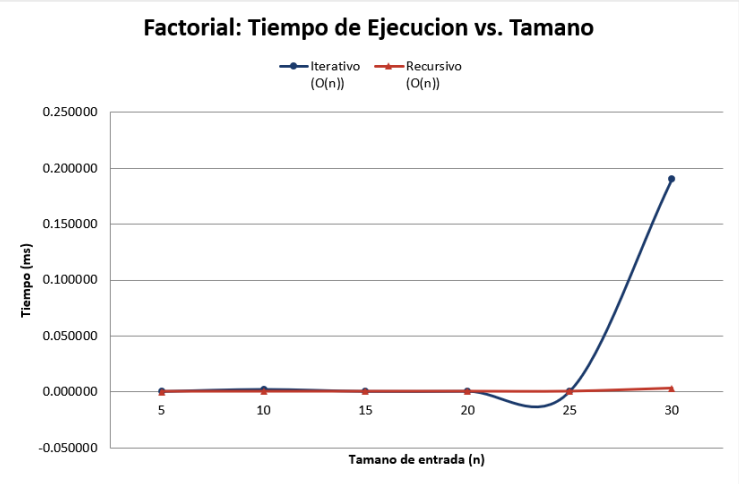
RESULTADOS

FACTORIAL

HOJA — A1 - Factorial | Iterativo vs. Recursivo		
Big-O Iterativo:	O(n)	Big-O Recursivo:

n	Iterativo (O(n))	Recursivo (O(n))
5	0.000262	0.000244
10	0.002140	0.000266
15	0.000364	0.000292
20	0.000448	0.000338
25	0.000504	0.000346
30	0.190640	0.003110

Observacion: Ambas versiones son O(n): el tiempo crece linealmente con n.

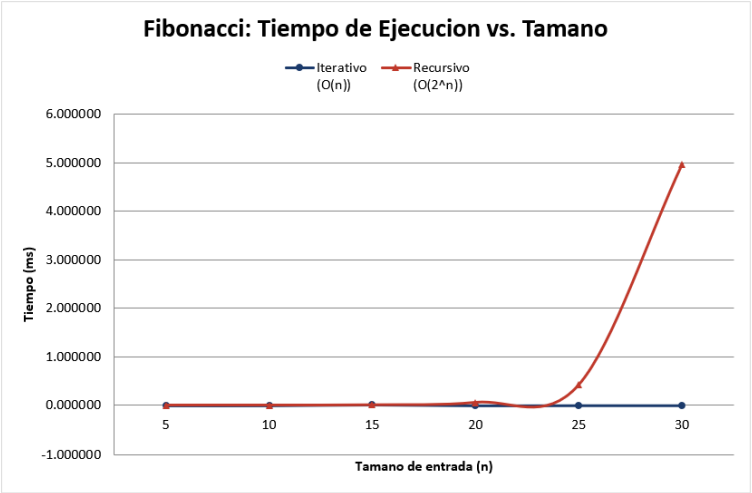


FIBONACCI

HOJA — A2 - Fibonacci | Iterativo vs. Recursivo		
Big-O Iterativo:	O(n)	Big-O Recursivo: O(2^n)

n	Iterativo (O(n))	Recursivo (O(2^n))
5	0.000288	0.000208
10	0.000562	0.000526
15	0.005652	0.005796
20	0.000478	0.055910
25	0.001310	0.424700
30	0.001160	4.979262

Observacion: Recursivo crece exponencialmente: duplicar n casi cuadriplica el tiempo.

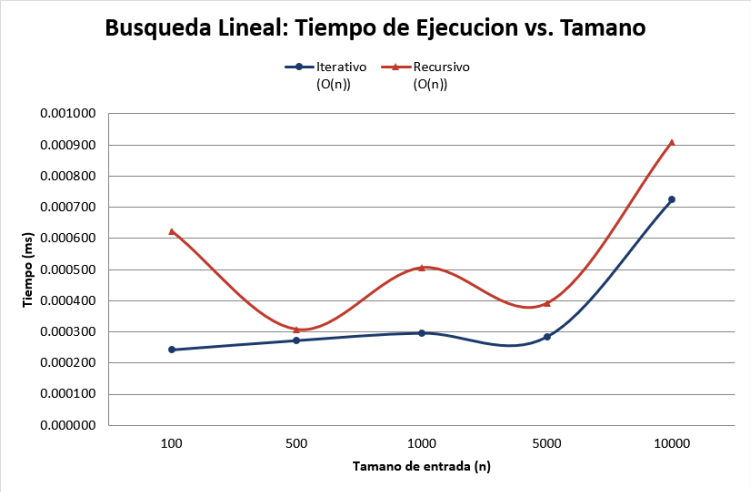


BUSQUEDA LINEAL

HOJA — A3 - Busqueda Lineal | Iterativo vs. Recursivo		
Big-O Iterativo:	O(n)	Big-O Recursivo:

n	Iterativo (O(n))	Recursivo (O(n))
100	0.000242	0.000624
500	0.000272	0.000308
1000	0.000296	0.000506
5000	0.000284	0.000392
10000	0.000724	0.000910

Observacion: Ambas O(n): al multiplicar n x10, el tiempo tambien x10 aprox.

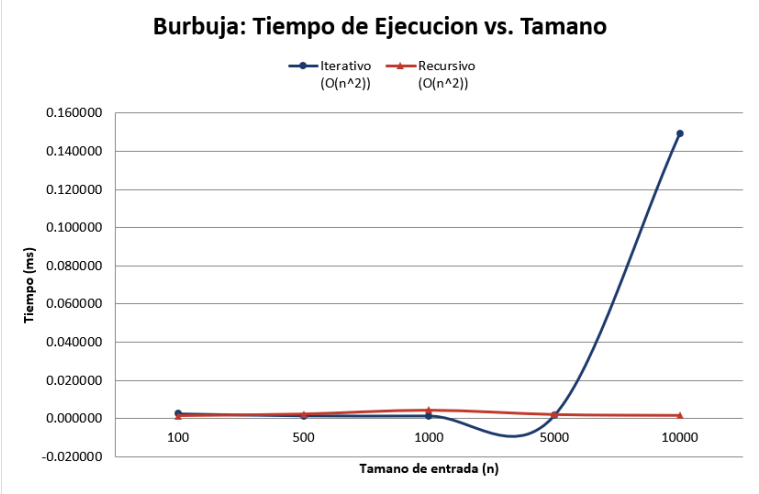


ORDENAMIENTO BURBUJA

HOJA — A4 - Burbuja | Iterativo vs. Recursivo		
Big-O Iterativo:	O(n^2)	Big-O Recursivo:

n	Iterativo (O(n^2))	Recursivo (O(n^2))
100	0.002574	0.001500
500	0.001542	0.002408
1000	0.001486	0.004550
5000	0.001714	0.002110
10000	0.149334	0.001628

Observacion: Ambas O(n^2): al x10 n, el tiempo aumenta aprox x100.



COMPARACIÓN ITERATIVO VRS RECURSIVO